

Technical Summary: Single-Source Shortest Paths with Negative Real Weights in $O(mn^{8/9})$ Time

Hussain Umer Farooqui, Hussain Mustansir

April 8, 2025

Problem and Contribution

The paper addresses the problem of computing single-source shortest paths (SSSP) on directed graphs with real edge weights, including both positive and negative values. The classic Bellman-Ford algorithm for this problem runs in $O(mn)$ time, where m is the number of edges and n is the number of vertices. This paper introduces a randomized algorithm that improves upon the Bellman-Ford algorithm by leveraging Dijkstra Algorithm, achieving a time complexity of $O(mn^{8/9})$, marking the first asymptotic improvement in the problem.

The main contribution of the paper is the development of a randomized algorithm that solves the SSSP problem for real-weighted graphs with high probability in the improved time complexity.

Algorithmic Description

Inputs

The input to the algorithm consists of:

- A directed graph $G = (V, E, w)$, where V is the set of vertices, E is the set of edges, and $w : E \rightarrow \mathbb{R}$ is a weight function assigning real weights (positive and negative) to the edges.
- A source vertex $s \in V$.

Outputs

The algorithm outputs:

- The shortest-path distances from the source vertex s to all other vertices in the graph, or
- It detects negative-weight cycles in the graph if there is a negative-weight cycle in the graph.

High-Level Logic of the Algorithm

The algorithm solves the Single-Source Shortest Path (SSSP) problem on graphs with negative real-weighted edges by following these key steps:

1. Betweenness Reduction:

- The first step is to perform **betweenness reduction**. This step involves creating a price function ϕ_1 that adjusts the weights of the graph, making negative edges less influential and simplifying the graph's structure. It prepares the graph for the next steps by reducing the complexity of paths affected by negative edges.

2. Finding a Negative Sandwich or Independent Set:

- Next, the algorithm tries to find either a **negative sandwich** (a subset of negative edges that can be isolated) or a **1-hop independent set** (a set of vertices with no negative edges between them). If it finds an independent set, it computes a price function to eliminate the negative edges in that set and returns the result.

3. Reweighting to Make Negative Edges Remote:

- If a negative sandwich is found, the algorithm continues by reweighting the graph to make the problematic edges **r-remote**. This means the edges are pushed far enough in the graph that they no longer affect the rest of the graph's structure. A price function ϕ_2 is applied for this reweighting.

4. Hop-Reduction to Eliminate Remaining Negative Edges:

- The final step is **hop-reduction**. This technique is applied to eliminate any remaining negative edges that were not removed by the previous steps. It uses the reweighted graph from the previous steps, and after this, the graph is ready for applying Dijkstra's algorithm.

The algorithm gradually eliminates or isolates negative edges, reweights the graph to avoid their influence, and then applies efficient algorithm like Dijkstra's on the transformed graph. This results in a much faster solution compared to the traditional Bellman-Ford approach, achieving a time complexity of $O(mn^{8/9})$.

Comparison with Existing Approaches

Bellman-Ford Algorithm

The Bellman-Ford algorithm is a well-known method for solving SSSP with real edge weights, running in $O(mn)$ time. This algorithm works by relaxing all edges iteratively. The new approach significantly improves upon this by reducing the time complexity to $O(mn^{8/9})$, using randomized techniques and graph reweighting.

Dijkstra's Algorithm

For graphs with non-negative edge weights, Dijkstra's algorithm achieves an optimal time complexity of $O(m + n \log n)$. The new algorithm, while designed for real-weighted graphs, handles negative weights efficiently while maintaining a reduced time complexity.

Novelty

This approach is novel because it reduces the impact of negative-weight edges via a probabilistic reweighting strategy and hop-limited paths, significantly improving on the time complexity compared to the classical Bellman-Ford algorithm.

Data Structures and Techniques

Hop-Limited Shortest Paths

The algorithm utilizes hop-limited shortest paths to efficiently handle graphs with negative-weight edges. By limiting the number of negative edges a path can contain, the algorithm reduces the number of possible negative edges that need to be considered in the graph.

Price Functions

The price function ϕ is used to reweight the graph to eliminate negative edges. This function is computed based on shortest-path distances, ensuring that the reweighted graph does not contain negative-weight edges, thus making it amenable to Dijkstra's algorithm.

Random Sampling

The algorithm uses random sampling techniques to identify sets of negative edges. This probabilistic approach helps reduce the time complexity of the algorithm by focusing on the most relevant edges and vertices in the graph.

Implementation Outlook

Challenges

The main challenges in implementing this algorithm are:

- **Large Input Size:** For graphs with a large number of negative-weight edges, ensuring that the algorithm runs within the expected time complexity could be challenging, especially when the number of iterations increases.
- **Randomization:** Ensuring that the randomization is implemented correctly and consistently will be crucial for achieving the desired time complexity.